PROGANADO S.A.S. — GUÍA MAESTRA DE SUSTENTACIÓN

Momento 2: Maquetación Web Profesional (React 19 + Tailwind CSS + Persistencia)

Institución: CESDE · Escuela de Tecnología e Innovación
Docente: Jonathan Felipe Piedrahita
Equipo: Jeiser Gutiérrez, Sebastián Correa, Emilio Villanueva, Humberto Pinto

🎯 **Propósito de esta Guía:** Proporcionar a cada integrante del equipo una explicación exacta y desglosada, **línea por línea en dos columnas** (Código de un lado vs Explicación del otro), dominando cada contenedor, div, formato de imagen, animación en CSS puro y el modelo de datos en el cliente vs bases de datos relacionales y NoSQL.

1. La Realidad de la Industria 2026: ¿Por Qué React + Tailwind y No Bootstrap?

En la sustentación, el equipo debe justificar con criterio de ingeniería por qué se migró de Bootstrap 5 a **React 19 + Tailwind CSS**:

Criterio Técnico	Bootstrap 5 (Legacy 2011–2020)	React 19 + Tailwind CSS (Estándar Industria 2026)
Adopción en la Industria	En declive para proyectos nuevos. Usado mayormente en mantenimiento de sistemas monolíticos heredados.	**Líder absoluto:** React domina con ~68% de cuota en SPAs; Tailwind tiene >80% de satisfacción en el *State of CSS*.
Paradigma de Diseño	*Componentes rígidos y prehechos:* Todas las webs se ven idénticas a menos que se sobreescriba con CSS pesado.	*Diseño atómico sin límites:* Clases utilitarias directas en el elemento sin salirse del flujo de diseño.
Rendimiento y Bundle	Carga hojas de estilo pesadas con cientos de clases que nunca se usan en la pantalla.	**Tree-shaking estricto:** Vite y Tailwind purgan el CSS y solo empaquetan las clases exactas que usamos (24 kB finales).
Alineación con el Docente	Tema superado en los primeros módulos de maquetación tradicional.	**Alineación total:** En la clase del 31-Oct, el docente Felipe Piedrahita introdujo `Tailwind + Flowbite` en clase.

2. Anatomía de la Página Web: Código JSX vs Explicación de Producción

A continuación se desglosan los bloques estructurales de la aplicación. En la columna izquierda se observa el código HTML/JSX exacto y en la derecha la explicación que debe dar el expositor.

Bloque 1: Header Institucional y Barra de Navegación (Navbar.jsx)

Código HTML / JSX en el Frontend	Explicación para el Jurado / Qué controla

```jsx
<header className="sticky top-0 z-40 bg-[#0b291b] border-b border-[#1f5038]">
  <div className="max-w-7xl mx-auto px-4 h-16 flex items-center justify-between">

    {/* Logo & Marca Ganadera */}
    <div className="flex items-center gap-3 cursor-pointer">
      <div className="w-10 h-10 rounded-xl bg-[#198754] flex items-center justify-center text-white font-black">
        PG
      </div>
      <h1 className="text-xl font-extrabold text-white tracking-tight">
        ProGanado<span className="text-[#75b798]">SaaS</span>
      </h1>
    </div>

    {/* Botones de Navegación */}
    <nav className="flex items-center gap-1 bg-[#133d2a] p-1 rounded-xl">
      {['dashboard', 'inventario', 'sanidad'].map((tab) => (
        <button key={tab} onClick={() => setActiveTab(tab)}
          className={activeTab === tab ? "bg-[#198754] text-white" : "text-[#a3cfbb]"}>
          {tab}
        </button>
      ))}
    </nav>
  </div>
</header>
```

1. Contenedor Semántico `<header>`:
`sticky top-0 z-40`: Hace que el menú quede fijado en la parte superior de la pantalla cuando el usuario hace scroll hacia abajo.
`bg-[#0b291b]`: Color Verde Esmeralda Profundo (#0b291b) que transmite identidad corporativa agropecuaria.
2. Container Central `max-w-7xl mx-auto`:
Evita que el contenido se estire infinitamente en monitores anchos (4K). Centra el contenido en 1.280 píxeles.
3. Navegación Dinámica Reactiva:
No recarga la página. El array `['dashboard', 'inventario', 'sanidad']` se renderiza con un `.map()`.
El botón activo cambia de color instantáneamente alternando clases de Tailwind mediante la variable de estado `activeTab`.

Bloque 2: Tarjetas de Bovinos, Contenedor de Fotos y Animación (InventoryView.jsx)

Código HTML / JSX en el Frontend	Explicación para el Jurado / Qué controla

```jsx
<div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-4 gap-5">
  {filtered.map((vaca) => (
    <div key={vaca.id} onClick={() => onSelectCow(vaca)}
      className="group rounded-2xl bg-white border border-[#e2ece6] overflow-hidden transition-all duration-300 hover:shadow-lg hover:-translate-y-1">

      {/* Contenedor de la Foto con Efecto Zoom */}
      <div className="h-44 w-full bg-[#f1f8f4] relative overflow-hidden">
        <img
          src={vaca.foto}
          alt={vaca.nombre}
          className="w-full h-full object-cover group-hover:scale-105 transition-transform duration-500"
        />
        <span className="absolute top-3 right-3 px-2
```

1. Grid Responsivo de Tarjetas:
`grid-cols-1 md:grid-cols-2 lg:grid-cols-4`: Maquetación adaptativa que se reorganiza automáticamente: 1 columna en celulares, 2 en tablets y 4 en PC.
2. Manejo y Formato de las Fotos (JPG locales):
`h-44 w-full relative overflow-hidden`: Contenedor con altura fija (176px). Si la imagen del ganado es más ancha o más alta, no deforma la tarjeta ni rompe la cuadrícula.
`object-cover`: Fuerza a la foto a llenar el contenedor recortando proporcionalmente los bordes (técnica usada en Instagram y Airbnb).
`src={vaca.foto}`: Lee el archivo local de `assets/ganado/vaca_XXXX.jpg`.
3. Micro-interacción y Animación de Movimiento:
`hover:-translate-y-1`: La tarjeta entera se eleva suavemente 4 píxeles al pasar el mouse por encima.

Código HTML / JSX en el Frontend	Explicación para el Jurado / Qué controla

`py-1 bg-black/60 backdrop-blur-md text-white text-xs rounded-lg">`
`{vaca.raza}`
`</span>`
`</div>`
 	
`{/* Datos del Bovino y Alerta ICA */}`
`<div className="p-4">`
`<h4 className="font-bold text-gray-900">{vaca.nombre}</h4>`
`<p className="text-xs text-gray-500">Chapa: {vaca.chapa}</p>`
`{vaca.diasRetiroIca > 0 && (`
`<span className="inline-block mt-2 px-2 py-0.5 bg-red-100 text-red-700 text-xs font-bold rounded">`
`⚠️ Retiro ICA: {vaca.diasRetiroIca} días`
`</span>`
`)}`
`</div>`
`</div>`
`))}`
`</div>`
	`group-hover:scale-105 duration-500`: La foto de la vaca hace un zoom suave de 5% sin salirse del marco, gracias al `overflow-hidden`.

Bloque 3: Animación Progresiva en CSS Moderno Puro (index.css)

Código CSS en src/index.css	Explicación para el Jurado / Qué controla

`/* Barra de Progreso de Lectura con CSS Scroll-Driven */`
`.scroll-progress-bar {`
`  position: fixed;`
`  top: 0;`
`  left: 0;`
`  height: 4px;`
`  width: 100%;`
`  background: #198754;`
`  transform-origin: 0% 50%;`
`  z-index: 100;`
 	
`  /* Propiedad Moderna W3C */`
`  animation: grow-progress auto linear;`
`  animation-timeline: scroll();`
`}`
 	
`@keyframes grow-progress {`
`  from { transform: scaleX(0); }`
`  to   { transform: scaleX(1); }`
`}`
	1. Cero Líneas de JavaScript (0% CPU en JS):
	Tradicionalmente se usaba `window.addEventListener('scroll', ...)` para calcular cuánto bajaba el usuario. Eso saturaba el procesador.
	Con `animation-timeline: scroll()`, el cálculo lo realiza el motor de renderizado del navegador directamente en la tarjeta de video (GPU).
	2. Cómo funciona la física del movimiento:
	`transform: scaleX(0)` a `scaleX(1)`: La barra empieza con ancho cero y se estira hasta el 100% a medida que se llega al footer.
	`transform-origin: 0% 50%`: Ancla el crecimiento al borde izquierdo de la pantalla.

3. Módulo Técnico: Arquitectura de Datos (Frontend vs SQL vs NoSQL)

Para sustentar con autoridad técnica, el equipo debe dominar las diferencias entre el almacenamiento en el cliente que usamos en la web, la base de datos relacional SQL que diseñamos para la materia de BD, y la base NoSQL (MongoDB) vista en clase con el docente.

Característica	1. Almacenamiento Frontend (localStorage)	2. SQL Relacional (SQLite / PostgreSQL / SQL Server)	3. NoSQL Documental (MongoDB)
¿Dónde vive?	En el navegador web de cada usuario (cliente).	En un servidor o archivo de base de datos relacional local/cloud.	En un servidor distribuido en formato documental.
Capacidad Máxima	**~5 MB a 10 MB** por dominio web.	**Gigabytes a Terabytes** (Ilimitado según el disco).	**Petabytes** (Escalabilidad horizontal masiva).
Estructura	Pares clave-valor en texto plano (Strings/JSON).	Tablas rígidas con columnas, tipos estrictos y DDL.	Colecciones de documentos BSON/JSON flexibles.
Integridad Referencial	Ninguna. Si borras un potrero, las vacas quedan huérfanas.	**Total (ACID):** Llaves foráneas (FK), ON DELETE CASCADE.	Por embebido o referencias manuales sin restricciones ACID rígidas.
Caso de Uso ProGanado	**Prototipo Web M2:** Guardar el estado de la sesión y filtros sin latencia de red.	**Núcleo Oficial (Bases de Datos):** 12 tablas con auditoría, pesaje de leche y trazabilidad sanitaria.	**Telemetría IoT:** Lecturas por segundo de collares GPS o sensores de temperatura en campo.

4. Cómo se Conecta la Página Web a la Base de Datos en Producción

Una página web de navegador **jamás se conecta directamente a una base de datos SQL** por razones de ciberseguridad (se expondrían las credenciales de la base de datos a cualquier usuario que abra la consola del navegador F12). El flujo estándar de la industria es una arquitectura en tres capas:

> **Arquitectura Estándar Cliente-Servidor:**
> `[Cliente: React Web] ⟷ (HTTP GET/POST / JSON) ⟶ [Servidor Backend: Node.js / Express] ⟷ (Query SQL) ⟶ [Base de Datos: SQLite / PostgreSQL]`

Implementación en el Código	Explicación del Proceso Paso a Paso

``// 1. EN EL FRONTEND (React)
// Hace una petición asíncrona a la API REST
async function cargarBovinosDesdeServidor() {
  const res = await fetch('http://localhost:3000/api/bovinos');
  const data = await res.json();
  setBovinos(data);
}
 	
// 2. EN EL BACKEND (Node.js + Express + SQLite)
// Recibe la petición y consulta la base de datos protegida
app.get('/api/bovinos', (req, res) => {
  const query = 'SELECT * FROM bovinos WHERE estado = "En Ordeño"';
  const rows = db.prepare(query).all();
  res.json(rows); // Devuelve los datos en formato JSON
});```
	1. Petición Asíncrona (Fetch API):
	React ejecuta la función `fetch()` en segundo plano. La interfaz del usuario no se congela mientras viajan los datos.
	2. Capa Intermedia de Seguridad (Backend REST API):
	El servidor backend actúa como un guardia de seguridad. Valida los permisos del usuario antes de tocar la base de datos.
	3. Ejecución del Query SQL:
	El backend ejecuta la sentencia `SELECT` contra la base de datos relacional y convierte las filas de la tabla en un array de objetos JSON para que React los pinte en las tarjetas.

5. Script de Sustentación Oral para el Equipo

> **Pregunta Clave del Profesor Jonathan:** *"¿Por qué su página web usa React y Tailwind en lugar de Bootstrap, y cómo manejan los datos?"*
>
> **Respuesta Recomendada del Expositor:**
> *"Profesor, en ProGanado tomamos la decisión de aplicar las tecnologías que hoy dominan la industria del software. Migramos de Bootstrap a React 19 con Tailwind CSS porque nos brinda arquitectura basada en componentes modulares, mayor velocidad de renderizado y un CSS optimizado con cero desperdicio de código.*
>
> *Respecto a los datos, implementamos una separación de responsabilidades: para este Momento 2, la web funciona como una SPA con persistencia en el `localStorage` del cliente y animaciones de scroll ejecutadas en GPU. Diseñamos este modelo en el frontend con exactamente los mismos atributos que definimos en las 12 tablas de nuestro modelo SQLite para la materia de Bases de Datos, dejando la aplicación 100% lista para conectarse mediante una API REST en el siguiente momento."*